Bitácora de defensa

SmartCity Analytics · Avance PostgreSQL + MongoDB · Cesar

Sesión 1 · 13 oct 2026 · Preparar el entorno

Resumen

Dejamos la computadora lista para el proyecto. Se instalaron WSL, Docker Desktop, DBeaver y MongoDB Compass (Python ya estaba), y se conectó el repositorio **SmartCity-Project-CE**. En el repo quedaron creados el **docker-compose.yml**, el **.env.example**, el **.gitignore** y el script **01_ddl.sql** que crea las 18 tablas.

Idea central: Postgres y Mongo no se instalan en Windows. Corren dentro de contenedores de Docker, así los 4 integrantes tienen exactamente el mismo entorno y todo se levanta con un solo comando.

Cómo se conecta todo

```
Tú (PowerShell)  →  docker compose up
                          │
                 Docker Desktop (usa WSL)
                  ├── contenedor Postgres  ←── DBeaver te deja verlo
                  └── contenedor Mongo     ←── Compass te deja verlo
                          ▲
              scripts de Python (llenan las bases con datos)

GitHub Desktop → guarda y comparte todos los archivos con el grupo
```

Herramientas y para qué sirven

Herramienta	Para qué la usamos	Ejemplo sencillo
WSL	Un Linux pequeño dentro de Windows. Docker lo necesita para funcionar.	Una segunda computadora con Linux escondida en la tuya; Docker la usa por debajo.
Docker Desktop	Corre Postgres y Mongo en contenedores, sin instalarlos en Windows.	Imagen = receta del pastel; contenedor = el pastel hecho. Misma receta, mismos pasteles.
Python	Scripts que generan y cargan los datos (100k+ filas, 500k+ documentos).	Un script de 30 líneas crea millones de mediciones en segundos.
DBeaver	Ver y consultar las tablas de PostgreSQL con interfaz gráfica.	Abres la tabla sensor como un Excel o ejecutas SELECT * FROM sensor;
MongoDB Compass	Ver y consultar las colecciones de MongoDB.	Ves cada documento como JSON: { "sensor": "S-00001", "pm25": 18.6 }
GitHub Desktop	Guardar versiones del código y compartirlo con el grupo.	Commit = guardar versión; push = subirla; pull = recibir la de otros.

Archivos del repositorio

Archivo	Para qué sirve	Ejemplo
docker-compose.yml	Lista de contenedores que Docker debe levantar y su configuración.	Postgres 16 en el puerto 5432 y Mongo 7 en el 27017, con un solo docker compose up.
.env.example / .env	Usuarios y contraseñas fuera del código.	El compose usa ${PG_PASSWORD} y Docker lo lee del .env, que nunca se sube.
.gitignore	Lo que git no debe subir.	El .env y los datos generados (pueden pesar GB) se quedan locales.
postgres/init/01_ddl.sql	Crea las 18 tablas de la base.	Docker lo ejecuta solo la primera vez que arranca Postgres.

Comandos clave

Comando	Qué hace
docker --version	Confirma que Docker está instalado.
docker run hello-world	Prueba que el motor de Docker corre (descarga y ejecuta una imagen de prueba).
docker compose up -d	Levanta Postgres y Mongo en segundo plano según el docker-compose.yml.
docker compose ps	Muestra qué contenedores del proyecto están corriendo.
docker compose down -v	Apaga los contenedores y borra los datos (para recrear las tablas).

Preguntas que pueden hacer en la defensa

¿Por qué usar Docker y no instalar PostgreSQL y MongoDB directamente?
Para que los 4 integrantes y el profesor tengan el mismo entorno, con las mismas versiones, y se levante con un comando. Evita el problema de "en mi computadora sí funciona".

¿Cuál es la diferencia entre una imagen y un contenedor?
La imagen es la plantilla (por ejemplo postgres:16). El contenedor es esa plantilla en ejecución. De una imagen se pueden crear varios contenedores iguales.

¿Por qué el archivo .env no se sube a GitHub?
Porque contiene contraseñas. Se sube solo el .env.example como plantilla y cada quien crea su copia local.

¿Cuándo se ejecuta el 01_ddl.sql?
Automáticamente, la primera vez que arranca el contenedor de Postgres con el volumen vacío. Si se cambia el DDL hay que usar docker compose down -v para que se vuelva a ejecutar.

¿Qué es un volumen de Docker?
Un espacio donde Docker guarda los datos fuera del contenedor. Si el contenedor se apaga o se borra, los datos siguen ahí.

Concepto clave · MongoDB y cómo se reparte el trabajo con PostgreSQL

¿Qué es MongoDB?

MongoDB es una base de datos **NoSQL de documentos**. En vez de guardar la información en tablas con filas y columnas, la guarda como **documentos JSON** dentro de **colecciones**.

PostgreSQL	MongoDB
Base de datos	Base de datos
Tabla	Colección
Fila	Documento (un JSON)
Columna	Campo
Todas las filas tienen las mismas columnas	Cada documento puede tener campos distintos

Ejemplo: dos sensores que miden cosas distintas

```
{ "sensor": "S-00001", "tipo": "calidad_aire",
  "ubicacion": { "zona": "Centro", "lat": 9.93, "lon": -84.08 },
  "medicion": { "pm25": 18.6, "co2": 425 },
  "timestamp": "2026-10-13T14:00:00" }

{ "sensor": "S-00002", "tipo": "trafico",
  "ubicacion": { "zona": "Norte", "lat": 9.95, "lon": -84.10 },
  "medicion": { "vehiculos": 132, "velocidad_prom": 34.5 },
  "timestamp": "2026-10-13T14:00:00" }
```

El primero tiene **pm25** y **co2**; el segundo, **vehiculos** y **velocidad_prom**. En Mongo conviven en la misma colección. En PostgreSQL habría que crear una columna por cada medición posible, la mayoría quedaría vacía y cada sensor nuevo obligaría a modificar la tabla.

Por qué lo usamos en el proyecto

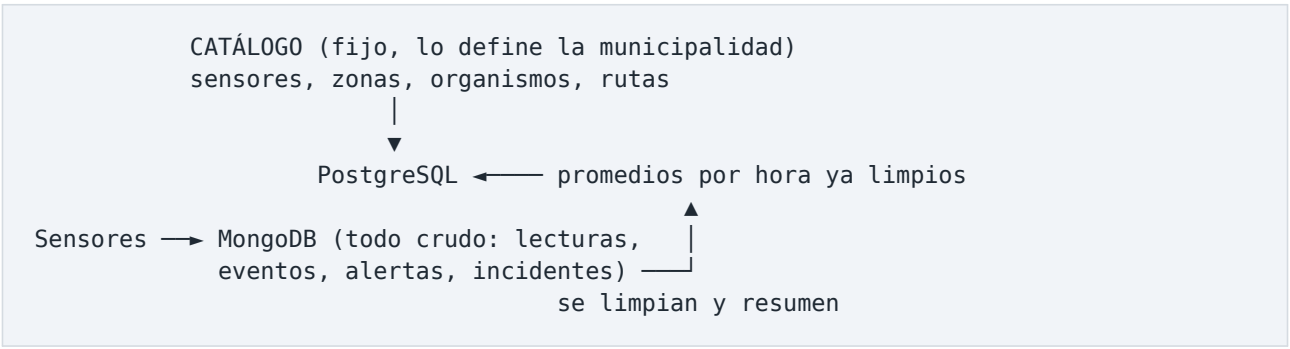
Los sensores IoT generan datos **semiestructurados**, en mucho volumen y muy seguido. Mongo encaja porque: (1) tiene **esquema flexible**, un sensor nuevo entra sin cambiar nada; (2) la ubicación va **embebida** dentro de la lectura, así que no hace falta JOIN; (3) es rápido para escribir grandes volúmenes (el proyecto pide 500.000+ documentos).

Cómo se consulta: Aggregation Framework

```
db.lecturas.aggregate([
  { $group: { _id: "$ubicacion.zona",
              promedio: { $avg: "$medicion.temperatura" } } }
])
```

Es una tubería de pasos por la que pasan los documentos: **$match** filtra, **$group** agrupa, **$sort** ordena. El ejemplo calcula la temperatura promedio por zona; equivale a un GROUP BY en SQL.

Cómo se reparten el trabajo MongoDB y PostgreSQL



	MongoDB	PostgreSQL
Qué guarda	Lo que mandan los sensores tal cual: lecturas, eventos, alertas, incidentes	El catálogo (sensores, zonas, organismos, rutas, vehículos) y los promedios por hora ya limpios
De dónde viene	De los sensores	El catálogo lo define la municipalidad; los promedios salen de los datos crudos
Qué aporta	Flexibilidad y volumen	Integridad, reglas y relaciones entre tablas
Se consulta	Sí: sensores sobre el límite, alertas por zona, etc. (consultas de D)	Sí: sensores por tipo, ranking de zonas, etc. (consultas de B y C)

El vínculo entre las dos bases es el id_sensor (por ejemplo S-00001): es el mismo código en ambas, así una lectura de Mongo se puede cruzar con los datos del sensor en Postgres.

En el proyecto completo, el paso de limpiar y resumir lo hará **Spark** (próximo avance). En este avance, los scripts de Python llenan las dos bases directamente.

Preguntas que pueden hacer en la defensa

¿Por qué usar MongoDB aquí?
No porque guarde JSON, sino porque las lecturas de sensores son semiestructuradas: cada tipo de sensor tiene campos distintos, y un modelo documental las guarda sin forzar un esquema rígido.

¿Por qué no guardar todo en MongoDB o todo en PostgreSQL?
Mongo guarda lo que llega de los sensores tal cual, con su flexibilidad. Postgres guarda el catálogo y los datos ya limpios, con integridad y relaciones. Cada uno se usa donde es más fuerte; el id_sensor las une.

¿Qué significa embeber un documento?
Guardar un dato dentro de otro documento en vez de en otra tabla. Por ejemplo, la ubicación dentro de la lectura: se lee todo junto sin JOIN.

¿Qué es el Aggregation Framework?
La forma de hacer análisis en Mongo: una tubería de etapas ($match, $group, $sort...) que transforma los documentos paso a paso, como un GROUP BY de SQL.

Sesión 2 · 15-16 oct 2026 · Bases arriba, contrato, Madrid y carga de datos

Resumen

Se levantaron PostgreSQL y MongoDB con **docker compose up -d** y se verificaron las 18 tablas. Se escribió el **contrato de datos**, se decidió usar **Madrid** como ciudad, se descargaron los datos abiertos reales y se escribieron los 4 scripts que llenan las dos bases.

Levantar las bases

Línea que muestra Docker	Qué significa
Image postgres:16 / mongo:7 Pulled	Descargó las imágenes (las "recetas")
Network ... Created	Red privada para que los contenedores se comuniquen
Volume pg_data / mongo_data Created	Espacio donde se guardan los datos aunque se apague el contenedor
Container smartcity_pg / smartcity_mongo Started	Las dos bases corriendo
0.0.0.0:5432->5432	El puerto de Windows lleva al Postgres del contenedor (así se conectan DBeaver y Python)

Si cambia el DDL hay que correr **docker compose down -v** (borra los volúmenes) y luego **up -d**, porque los scripts de **postgres/init** solo se ejecutan con el volumen vacío.

Cómo leer el SQL del DDL

```
CREATE TABLE vehiculo (
    id_vehiculo INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    placa       VARCHAR(15) NOT NULL UNIQUE,
    tipo        VARCHAR(15) NOT NULL CHECK (tipo IN ('bus','auto','moto','bicicleta')),
    capacidad   SMALLINT CHECK (capacidad > 0),
    id_ruta     INT REFERENCES ruta(id_ruta)
);
```

Parte	Significado	Ejemplo
GENERATED ALWAYS AS IDENTITY	Postgres pone el número solo (1, 2, 3...)	No se inventa el id
PRIMARY KEY	Llave primaria: identifica la fila, no se repite ni queda vacía	Vehículo 7 es único
NOT NULL / UNIQUE	Obligatorio / no se puede repetir	Dos vehículos no comparten placa
CHECK (...)	Regla que el dato debe cumplir	tipo = 'avion' se rechaza
REFERENCES ruta(id_ruta)	Llave foránea: el valor debe existir en la otra tabla	No hay bus en la ruta 99 si no existe

Parte	Significado	Ejemplo
PARTITION BY RANGE (fecha_hora)	La tabla medicion se divide por año	Una consulta de 2025 solo lee medicion_2025 (partition pruning)

Contrato de datos

Es un acuerdo escrito (**docs/contrato_datos.md**) sobre cómo se llama y cómo se ve cada dato, para que lo que genera uno calce con las consultas de los demás. Define: formato del id_sensor (S-00001), zona = distrito, tipos de sensor y variables con su límite de alerta, organismos, qué va en cada base y el formato de los JSON de Mongo.

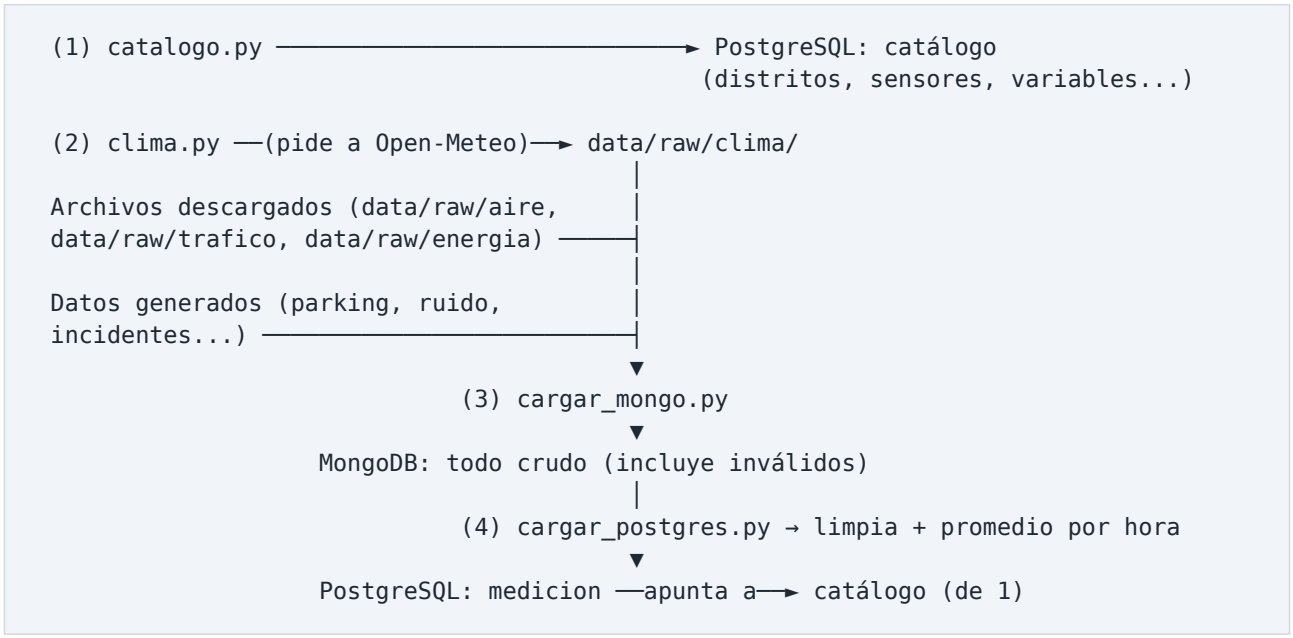
Por qué Madrid y no San José

	San José	Madrid (elegida)
Tráfico	Patrón de otra ciudad	Real: 7.000+ detectores cada 15 minutos
Calidad del aire	Estimación de modelo (~45 km)	Real: 24 estaciones, valores por hora con validación
Volumen	Habría que inflarlo	Real: solo junio 2026 tiene 13,3 millones de filas de tráfico
Para Hadoop/Spark	Big Data artificial	Big Data real que justifica procesamiento distribuido

Datos descargados

Archivo	Qué trae
Calidad del aire 2024 y desde 2025	Por estación, magnitud y día: 24 columnas de horas (H01..H24) y su validación V/N
Estaciones de control	24 estaciones con coordenadas, tipo y fecha real de alta
Tráfico 06, 07 y 08 de 2026	id, fecha, intensidad, ocupacion, carga, vmed, error cada 15 min (2,3 GB)
Ubicación de puntos de tráfico	5.083 puntos con distrito y coordenadas: conecta el id del tráfico con la zona
UCI Household Power	Consumo eléctrico por minuto (2 millones de filas): solo se usa su patrón semana/hora

Los 4 scripts y el orden



Script	Qué hace	Por qué en ese orden
catalogo.py	Llena las tablas fijas de Postgres: distritos, ubicaciones, sensores, variables, organismos, rutas, vehículos	Sin sensores creados, una medición no tiene a quién apuntar (llave foránea)
clima.py	Pide a Open-Meteo el clima por hora de cada distrito y lo guarda en data/raw/clima	cargar_mongo.py necesita esos archivos
cargar_mongo.py	Mete todo lo crudo a Mongo (lecturas) y calcula alertas y eventos; genera incidentes	Los sensores mandan todo a Mongo primero
cargar_postgres.py	Lee de Mongo, descarta inválidos, promedia por hora y llena medicion	Es el paso de limpieza; en el avance de Spark lo hará Spark

Limpieza con el Aggregation Framework

```
[ {"$match":   {"tipo": "trafico"}},
  {"$project": {"id_sensor": 1,
                "hora": {"$dateTrunc": {"date": "$timestamp", "unit": "hour"}},
                "kv":   {"$objectToArray": "$medicion"},
                "inv":  {"$ifNull": ["$invalidos", []]}}},
  {"$unwind":  "$kv"},
  {"$match":   {"$expr": {"$and": [{"$not": [{"$in": ["$kv.k", "$inv"]}]},
                                   {"$isNumber": "$kv.v"}]}}},
  {"$group":   {"_id": {"s": "$id_sensor", "h": "$hora", "k": "$kv.k"},
                "v":   {"$avg": "$kv.v"}}} ]
```

Cada etapa transforma los documentos: corta la hora, separa cada variable, descarta las inválidas y promedia. Ejemplo: lecturas de tráfico a las 8:00, 8:15, 8:30 y 8:45 con intensidad 100, 300, 200 y 200 quedan en Postgres como una sola fila: 8:00 = 200.

Resultado de la carga (16 oct 2026)

Base	Colección / tabla	Registros	Mínimo del enunciado
PostgreSQL	sensor	324 (304 activos, 11 inactivos, 9 en mantenimiento)	-
PostgreSQL	medicion	4.664.093	100.000
PostgreSQL	medicion_2024 / 2025 / 2026	1.089.649 / 1.281.661 / 2.292.783	-
MongoDB	lecturas	3.151.581	500.000
MongoDB	alertas / eventos / incidentes	47.443 / 7.397 / 4.000	-

En total son casi 8 millones de registros, por encima del escenario recomendado (más de 5 millones). Postgres tiene más filas que Mongo porque cada lectura de Mongo trae varias variables y en Postgres cada variable es una fila; en cambio el tráfico baja de 1,75 millones de lecturas cada 15 min a 1,35 millones de filas (un promedio por hora y por variable).

Preguntas que pueden hacer en la defensa

¿Qué es un contrato de datos y para qué lo hicieron?
Un acuerdo escrito de nombres y formatos. Sirve para que los cuatro integrantes trabajen con los mismos datos y para que PostgreSQL y MongoDB se puedan cruzar por el id_sensor.

¿Por qué Madrid?
Porque publica datos abiertos reales de tráfico y calidad del aire de la misma ciudad y las mismas fechas, con volumen real: eso permite medir la relación tráfico-contaminación y justifica Big Data en los siguientes avances.

¿Qué datos son reales y cuáles generados?
Reales: estaciones y valores de calidad del aire, puntos y lecturas de tráfico, clima. Generados (con semilla fija y documentados en docs/generacion_datos.md): organismos, rutas, vehículos, energía, parking, ruido, incidentes y estados.

¿Por qué Mongo guarda también los datos inválidos?
Porque es el almacén crudo: conserva todo tal como llegó, marcado con valido/invalidos. Postgres solo recibe lo limpio. Así se puede auditar o volver a procesar sin perder información.

¿Por qué no cargaron los 40 millones de filas de tráfico?
Para este avance bastan 210 puntos (unos 1,7 millones de lecturas). El volumen completo se usará en el avance de Hadoop, donde justamente se necesita almacenamiento distribuido.

¿Qué es la llave foránea y qué pasa si se viola?
Una columna que debe existir en otra tabla. Si se intenta guardar una medición de un sensor que no existe, Postgres la rechaza.